TASK 1

1. 查找**模型预训练**与**模型微调**相关知识，并了解其好处：

模型预训练：在大规模通用数据集（比如 ImageNet，千万张图片）上完整训练一个 CNN（如VGG等），

让网络自动学习图像的通用特征（底层：边缘、线条、色彩块 /中层：纹理、形状 /高层：物体轮廓、整体类别）

训练后保存下来的权重作为预训练权重

微调：拿已经训练好的预训练模型，根据新的任务数据继续训练模型，使模型适应具体任务

好处：

- 让小数据集也可训练好模型（假如数据集小，从零开始会导致过拟合等问题，先训练出通用特征再进行少量微调即可让小数据集也可训练好模型）
- 提升了模型性能
- 不需要从零训练大量图像，节省时间和成本
- 泛化能力好（因为大部分权重取自通用特征，所有适配到新任务效果好

2. 了解***冻结参数***的作用，模型微调时超参数该如何设置：

作用：冻结主干网络，只调节新加的输出层，从而保护预训练后得到的底层通用特征，防止小数据集训练微调高层权重时破坏底层基础权重，以导致模型忘记纹理、线条等基本特征识别知识。

超参数设置：取很小的学习率（如1e-5）；少训练轮次（几十轮即可，loss一旦收敛就可以早停防止过拟合）；少图像批次（10次左右）；正则化的损失函数惩罚调大一些（防止过拟合）

3. 在风格迁移领域预训练模型有什么作用：

起到多特征提取器的作用，提供了内容和风格的衡量标准（并且不用从零训练CNN）

- ***高层特征***（如图像内容、页面布局）：用来计算内容损失，保证生成图保留原图的主体结构
- ***低层特征***（如线条、纹理、色彩）：计算Gram矩阵，用来计算风格损失

TASK 2

1. 风格迁移需要哪两张图片？它们分别起什么作用？

- ***内容图片***：提供生成图片的主要结构（事物类别、布局）
- ***风格图片***：提供生成图片的主要风格（笔触、色彩、纹理）

2. 在神经网络中，图像的“**内容特征**”和“**风格特征**”是如何提取的？

- ***内容特征***：VGG高层（VGG19的第25层--还保留很强的空间信息，适合做内容特征）
- ***风格特征***：主要来自浅层和中层，通过计算不同特征图之前的相关关系来获得风格信息（VGG19的第

[0,5,10,19,28]层)

注意：浅层和深层不是绝对的。在最适合作内容特征的层以后，池化等操作反而会压缩图片的空间细节，凸显局部纹理和色彩相关性；因此内容图片不会取最深层反而风格图片会有取到。

3. **卷积神经网络（CNN）**在风格迁移中扮演了什么角色？特征图、卷积核、步长在这里有何意义？

CNN：用预处理好的模型（通常为VGG19），不改变各层卷积层的参数，只用于提取特征

特征图：作为卷积的输出，代表着图像的一个特征（颜色、纹理等等）

卷积核：提取特征的小窗口。与普通卷积的意义一致(?)通过平移移动对该窗口内的像素按权重加权求和（各通道再求一次和）得到输出图像

步长：卷积核每次移动的距离。风格迁移中步长代表了对特征检测的精细程度，并且影响着输出图像的大小

4. **风格损失、内容损失、总变分损失**分别代表什么？它们在生成结果时起到怎样的平衡作用？

内容损失：衡量生成图像与内容图片的内容接近程度。loss越小，生成图与内容图像内容越接近

风格损失：衡量生成图像与风格图片的风格接近程度。loss越小，生成图与风格图像风格越接近

总变分损失：衡量生成图像的图片平滑程度。loss越小，生成图像的噪声、不自然的小纹理越少

平衡作用：总loss=a * 内容loss+b * 风格loss+c * 总变分loss （a,b,c为权重），使生成图像尽可能同时满足三者

5. 优化过程是如何一步步让生成图像接近目标效果的？最终模型会输出什么？

```
1. 准备一张内容图片、风格图片，同时初始化一张随机照片（用内容图片or随机噪声初始化）
                ↓
2. 将随机照片输入进预训练好的CNN模型（内容图片、风格图片同）
                ↓
3. 模型提取图片的内容特征和风格特征（同时内容图片提取内容特征、风格图片提取风格特征并储存）
                ↓
4. 计算内容损失、风格损失（利用gram矩阵）、总变分损失，最后加权得到总损失
                ↓
5. 使用梯度下降更新图片像素，生成新图片
                ↓
6. 不断迭代，直至损失降至最低，最后得到兼具内容与风格的图像
```

最后输出：一张融合内容图片和风格图片特点的新图像

TASK 3

1. 如果所选的风格图片和内容图片尺寸相差太大，会有影响吗？

会有影响。比如512×512的内容图片和256×256经过卷积会分别生成64×64和32×32的特征图，两者尺寸不同不能直接运算。

解决方法：加入transforms.Resize((512,512))代码，将图片统一调成512×512像素大小

2. 请简要了解**内容损失、风格损失、全变分损失**的数学原理，并用自己的话进行解释。

（解释会略显通俗些，这里就不列出公式了都已经写在代码里了，主要就是方便理解）

- ***内容损失***：分别拿生成图和内容图经过vgg时较为高层的feature_map（如本题的conv4_2,就是经过论文论证过的比较能表现图像中物体空间结构的特征图）,对其进行均方损失（特征图--即矩阵每个元素相减后平方再相加）即得到loss，用来衡量生成图像以内容图片为参照的物体偏移程度
    - ***风格损失***：分别拿生成图和风格图经过vgg时较为低层的特征图（不绝对，我是直接参照那篇经典论文得出的最能展现特征的那几个层，如'conv1_1','conv2_1','conv5_1'等），对特征图生成gram矩阵（不同特征的相关性图像，显示不同特征组合起来的风格信息），对生成图和风格图的各自gram矩阵做均方损失（矩阵每个元素相减后平方再相加）即得到loss，用来用来衡量生成图片与风格图片风格相差程度
    - ***全变分损失***：拿还没进入vgg的生成图的tensor形式([1,3,516,516])作为对象，进行 除去最后一行的矩阵 **-** 除去第一行的矩阵 的操作，再对结果的每个元素取绝对值后相加得到行的变分损失；列的变分损失进行相同操作后，行列损失相加即得到全变分损失（理解：其实就是算矩阵里的每个元素与周围各个元素的差值的总合，该差值用来衡量色差的平滑程度---就是让图片有较少突变的杂点和不平滑的地方）

3. 风格损失中**gram矩阵**的作用是什么？为什么内积能表示两个通道间的相似度？

作用：计算不同特征通道之间的相关性，由此来描述不同特征组合起来的风格信息

原因：gram=torch.mm(features,features.t())

feature结构: $([c_1],[c_2],\cdots)^{\mathrm{T}}$,每个通道里是被一维化的特征图像；

将feature矩阵乘他的转置得到gram，gram里的（i，j）元素就表示第i个通道向量和第j个通道点乘后的结果。一维化忽略了特征的空间位置，而后通道间的点乘可以更好的表示两通道间的相似度，由此gram很好地关注了图像的整体风格

4. 请比较风格损失与内容损失的差异，为什么**gram矩阵**更适合捕捉风格？

差异:

- 内容损失关注图片结构，保留了位置信息，最后是通过生成图和内容图片的特征图进行相减均方算loss；
- 风格损失关注各特征之间的关联程度，丢弃了位置信息，最后是通过生成图和风格图片的gram(本质上就是特征关联图)相减均方算loss

gram矩阵更适合捕捉风格原因：相较于内容特征，gram矩阵丢弃了位置信息，而保留和生成了对应位置（位置本身重要，重要的是一一对应）的特征和特征间的关系，比如图像（星空）蓝色和粗条纹的关联度很高，从而可以较好捕捉到图像的风格

5. 在计算内容损失和风格损失时，为什么通常要将**内容图和风格图对应的目标特征**从计算图中分离？

假如不分离，梯度路径会将内容图和风格图对应的目标特征也当作计算图中的变量而非固定指标；这样反向传播时计算图就会尝试计算以该目标为变量下的梯度（整个就错乱了），因此我们需要将其从计算图中分离作为一个固定常数指标。

（其实不分离问题也不大，只要在优化器部分指定以生成图作为真正参量即可，分离后实际的好处是节省显存）

6. 使用迁移学习来提取特征时应设置为评估模式，这和冻结参数是否是一回事？

不是一回事。

冻结参数只是让参数不更新，也就是让vgg不学习；而***设置为评估模式***则是改变了模型的状态，让模型在运行时关闭**Dropout**（功能：训练时自动关闭一些神经元，防止神经网络过度依赖某几个神经元；此时并非训练模型所以不需要）和**BatchNorm**（功能：批归一化，在模型训练时遇到多批次时对前后批次同时更新运行平均值；当前只有一批次所以批归一化没有意义，为了输入同一张图片得到稳定特征需要让BatchNorm固定）

7. 如何从模型中抽取合适的层作为内容层和风格层，这样做的依据是什么？

依据：VGG不同层提取的信息不同。

方法：根据论文已经给出的结论，选取conv1_1，conv2_1，conv3_1，conv4_1，conv5_1作为风格层提取纹理、颜色等风格信息；选取conv5_2作为内容层提取内容信息（具体操作：将tensor图进行for循环一层层的跑下去，跑到conv对应的层序号后把feature_map（x）抽取出来存在字典里）

8. 风格迁移任务中适合使用L-BFGS优化器，相较于**Adam、SGD**的特点是什么？（当然，你可以选择后两种方法，通过实际训练挑选出来最适合该任务的优化器）

普通神经网络需要优化百万个参数，而风格迁移只需要优化一张图片的像素。L-BFGS相较于Adam、SGD梯度收敛速度快、精度更高，适合做连续变量的优化(连续变量：即像素的RGB值变化是连续的，并非只能以整数形式更新)

数学理解：L-BFGS拟牛顿法利用了矩阵的二阶导曲率信息，矩阵的二阶导的转置×一阶梯度，对目标（图像）比较"光滑"的做出改变（这就是为啥那么要让变量连续的原因），但前提是变量较少，不然二阶导的数据会非常大，求值太慢了。

（切身体会到该优化器的便捷，因为每次梯度下降都会自动尝试好几个方向找到最快方向，正常只用跑250个epoch就基本收缩）

9. 简要总结一下风格迁移主循环的流程。

```
输入内容图和风格图
        ↓
使用VGG19提取特征
        ↓
初始化生成图
        ↓
循环：

    提取生成图特征
        ↓
    计算损失：拿生成图特征和内容图风格图特征计算损失
        ↓
    反向传播计算梯度
        ↓
    使用优化器更新生成图像像素
```

```
        ↓
输出最终兼具内容和风格的艺术图片
```

10. 简述风格迁移中**前向传播和反向传播**的流程。

前向传播：

```
生成图：
↓
输入至VGG19
↓
得到feature
↓
计算：内容损失、风格损失、TV损失，加权得到总损失
```

反向传播：

```
由损失结果反向传播，计算梯度，得到每个像素应该如何改变
        ↓
优化器更新，使损失下降
```

11. 风格迁移任务中有哪些超参数？各个损失间的权重该如何设置？

- ***epoch***：可以先试运行一次看看损失在大概多少轮次时基本收敛，之后训练就可以定比这个稍微多一点的轮次作为epoch
- ***lr***：这个直接使用了L-BFGS的默认学习率，理论上在定义优化器时可以调节但没啥必要感觉
- ***各个损失合成总损失时对应的a,b,c权重***：超级超级难调！做这个任务时花最多的时间就是在调着叁上。**理论上**是要求损失收敛后的内容损失×权重a 与 风格损失×权重b差不多相等，而实际过程中因为优化器总是想着总损失趋于最小 + 每一次尝试等待收敛的时间很久把这个作为主要判断方法根本行不通。**我的实际操作**：先问ai让他给我经典的参数大致量级，再通过生成图片的样子结合权重大小对图片的**影响**调节权重量级不断尝试直到生成的图片满意。（内容损失对应的a权重占比越大，生成图的内容结构长得跟内容图片越像；风格损失对应的b权重越大，生成图的风格就越参照风格图片；而tv损失对应的权重c太大会导致生成图片止学习如何让图片变“顺滑”而只在内容图片的基础上把图片变模糊，太小会导致生成图的杂点太多----------然而不是三个都越大越好，这里强调的是占比/比例，三个都同时大了只会徒增梯度让优化器运算爆炸），根据这套关系慢慢调节才能得到比较优秀的生成图片。

12. 模型的总损失越小迁移效果越好吗？在合成图片过程中你是如何调参的？

不是。损失只是一个数学指标，我们只注重其前后的相对大小变化。过度减小某个损失权重只会让图片发生我上一题说过的问题；而所有权重集体减小/增大只会让梯度过小/过大

how：调参方法我上一题说过，就是根据图片的实际效果“缺哪补哪”进行调参，直至生成图片满意为止。

PS 对自己的特殊操作进行说明：原本求gram矩阵时用到了gram/(c×h×w)，但实际调试参数时发现风格特征被抑制严重实际效果并不好。最后与ai进行讨论后得出可以不按经典的教程/c的走（反正这只是一个通常做法，对数学原理并没有造成曲解和影响），最终没有/c只有gram/(h×w)来对gram矩阵进行标准化（效果立竿见影）

最后附上代码和图片效果

代码：

```python
import torch
import torchvision.models as models
from torchvision.models import VGG19_Weights
from PIL import Image
import torchvision.transforms as transforms
import warnings

warnings.filterwarnings("ignore")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
#1. 数据准备
content_img = Image.open("content.jpg").convert("RGB")
style_img = Image.open('style.jpg').convert("RGB")
transform = transforms.Compose([
    transforms.Resize((512,512)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406],std=[0.229,0.224,0.225])
])
content_img=transform(content_img)
style_img=transform(style_img)
content_img=content_img.unsqueeze(0).to(device)
style_img=style_img.unsqueeze(0).to(device)


#2. 加载预训练模型
vgg = models.vgg19(weights=VGG19_Weights.DEFAULT).to(device)
for param in vgg.parameters():
    param.requires_grad=False
vgg.eval()


#3. 定义特征提取工具
def get_features(image,model):
    features={}
    x=image
    layers={
        '0':'conv1_1',
        '5':'conv2_1',
        '10':'conv3_1',
        '19':'conv4_1',
        '21':'conv4_2',
        '28':'conv5_1'
    }
    for name,layer in model.features._modules.items():
        x=layer(x)
        if name in layers:
            features[layers[name]]=x
    return features

def gram_matrix(tensor):
    b,c,h,w=tensor.size()
    features=tensor.reshape(c,h*w)
    gram=torch.mm(features,features.t())
```

```
        return gram/(h*w)       #原本要多除以c，是为了让gram矩阵整体稳定，跟数学推导无关


#4. 定义损失函数
def content_loss_f(generated,content):
    return torch.mean((generated-content)**2)

def style_loss_f(generated,style):
    G1=gram_matrix(generated)
    G2=gram_matrix(style)
    return torch.mean((G1-G2)**2)

def tv_loss_f(img):
    loss1=torch.sum(torch.abs(img[:,:,:-1,:]-img[:,:,1:,:]))
    loss2=torch.sum(torch.abs(img[:,:,:,:-1]-img[:,:,:,1:]))
    return loss1+loss2

def total_loss_f(content,style,tv):
    a = 1
    b = 150
    c = 1e-6
    return a*content+b*style+c*tv



#5. 初始化生成图
#gen_img=torch.randn_like(content_img).requires_grad_(True)
gen_img = content_img.clone().detach().requires_grad_(True) #或采用随机噪声初始化generated_img=torch.randn_like(content_img).require_grad_(True)


#6. 找到内容图、风格图的特征
with torch.no_grad():
    content_features = get_features(content_img, vgg).detach()
    style_features=get_features(style_img,vgg).detach()
content_features=content_features['conv4_2']

#7. 选择优化器
optimizer = torch.optim.LBFGS([gen_img],lr=0.2,max_iter=20)


#8. 训练循环
def closure():
    optimizer.zero_grad()
    style_loss=torch.tensor(0.,device=gen_img.device)
    gen_features=get_features(gen_img,vgg)
    tv_loss=tv_loss_f(gen_img)
    for feature in ['conv1_1','conv2_1','conv3_1','conv4_1','conv4_2','conv5_1']:
        if feature=='conv4_2':
            gen_content_features=gen_features[feature]
            content_loss=content_loss_f(gen_content_features,content_features)
        else:
            style_loss+=style_loss_f(gen_features[feature],style_features[feature])
    total_loss=total_loss_f(content_loss,style_loss,tv_loss)
    total_loss.backward()
    print(f"Epoch {epoch} | content: {content_loss.item():.2f} | style: {style_loss.item():.2f} | tv: {tv_loss.item():.4f}")
    return total_loss

for epoch in range(300):
    loss = optimizer.step(closure)
    if (epoch+1)%2==0:
        print(epoch+1,loss.item())


#9. 输出最终图像
def unnormalize(img):
    mean = torch.tensor([0.485,0.456,0.406],device=img.device)
    std = torch.tensor([0.229,0.224,0.225],device=img.device)
    mean = mean.reshape(1,3,1,1)
    std = std.reshape(1,3,1,1)
    return img*std+mean

final_img = unnormalize(gen_img).clamp(0,1)
final_img = final_img.squeeze(0)
img = transforms.ToPILImage()(final_img)
img.save("output.jpg")
```

效果：

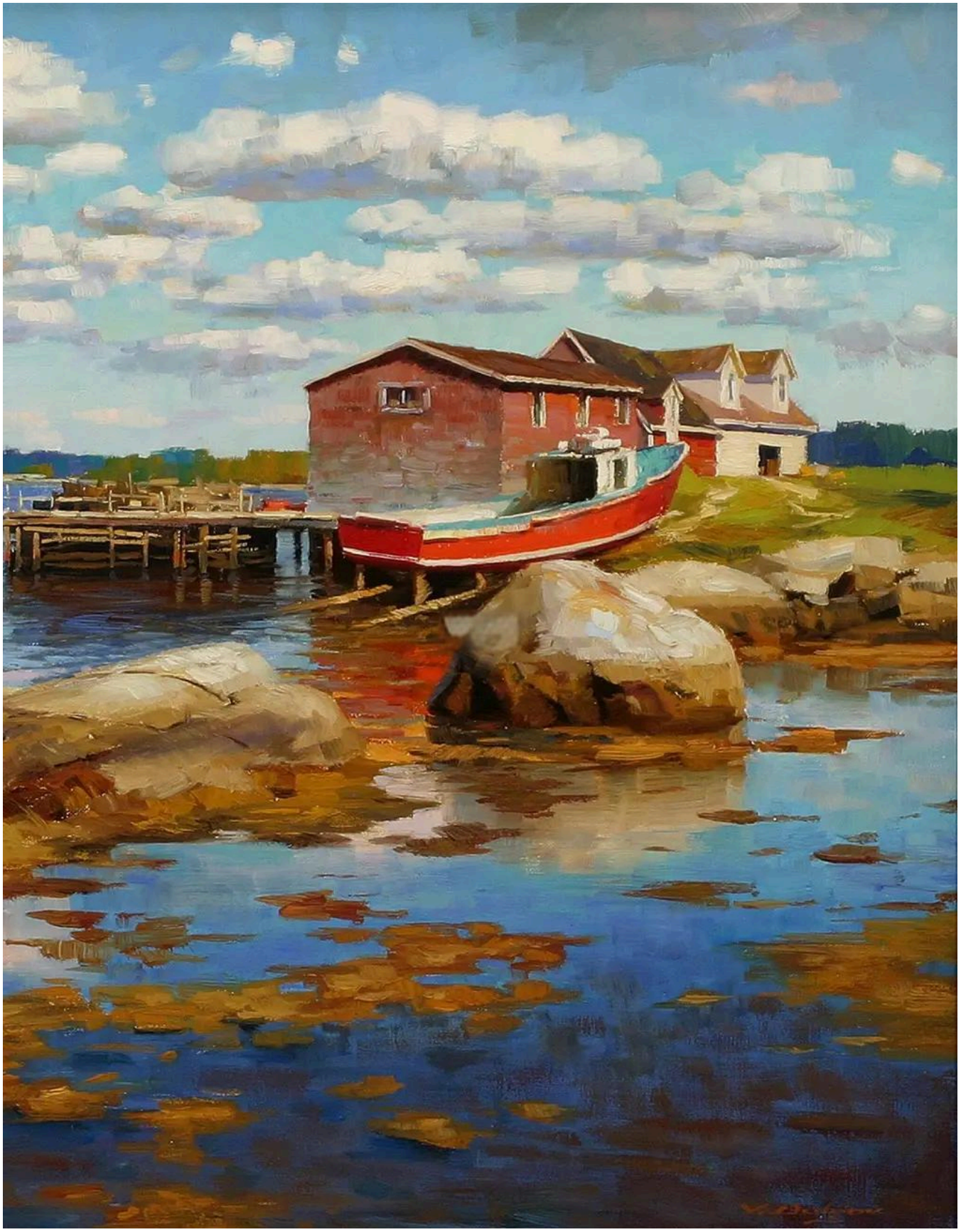
内容图片：



风格图片（用了三张风格图片，让其分别生成了不同风格的生成图）：







效果图（p1，p2由第一张风格图片产生，用了不同参数生成；剩下的两张一对一）：

PS: 个人认为最后一张效果图最好看







